More practice - Intervals and scheduling

4 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1	3
MEDIUM	HARD

How to use

- Questions run from Easy to Medium to Hard, newest last-asked date first inside each level.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

More practice - Intervals and scheduling | 4 questions | tap a title to jump to it

M	1. Simulate room assignments for scheduled meetings	PracHub	2026-01-06
H	2. Minimize travel time with optional meeting point	PracHub	2026-01-06
H	3. Compute city skyline outline	PracHub	2025-12-08
H	4. Compute Turnstile Crossing Times	PracHub	2026-05-04

STUDY PATH: question 114 of 167 (more practice)

Priority score 3.1 | Intervals and scheduling | asked 287 days ago

MEDIUM | IN APP LC 2402 | PracHub | Technical Screen | 1 of 4

Simulate room assignments for scheduled meetings

POSTED 2026-01-06 | 287 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 40 | 30 DAYS 0 | PRIORITY 95/228

ABOUT THIS QUESTION

Topic: More practice - Intervals and scheduling

Optimal answer from: your app's interview approach (LeetCode #2402)

LeetCode match: LC 2402 Meeting Rooms III (inferred from overview)

Posted on PracHub: 2026-01-06 (updated 2026-03-29)

Original: https://prachub.com/interview-questions/simulate-room-assignments-for-scheduled-meetings

QUESTION

Overview: This question evaluates scheduling and event-simulation skills, including time-ordered processing, resource allocation with tie-breaking, and efficient tracking of room availability and usage counts using appropriate data structures.

APPROACH

Sort meetings by start, min-heap of free rooms (by index) and a heap of busy rooms (end time, room); assign the lowest free room, else wait for the earliest finish and delay.

HOW TO THINK ABOUT IT

Simulate in start order with two questions per meeting: which rooms are free now (lowest index first), and if none, which room frees up first (the meeting is delayed but keeps its length). Two heaps answer both quickly: one of free room ids, one of (end time, room id) for busy rooms.

Basis: LeetCode 2402, Meeting Rooms III.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(n, meetings):
    end = [0] * n
    count = [0] * n
    for start, finish in sorted(meetings):
        free = [r for r in range(n)
                if end[r] <= start]
        if free:
            r = min(free)
            end[r] = finish
        else:
            r = end.index(min(end))  # earliest
            wait = end[r] - start
            end[r] = finish + wait
        count[r] += 1
    return count.index(max(count))
```

TIME	SPACE
O(M * N)	O(N)

WHY IT IS SLOW

Every meeting scans all N rooms, O(M * N). Two heaps (free rooms and busy rooms by end time) make each meeting O(log N).

OPTIMAL SOLUTION · Two heaps: free rooms and busy rooms

TIME	SPACE
O(M log M + M log N)	O(N)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if meetings arrive online and cannot be reordered?

The same two heaps work if you process meetings in arrival order and treat the arrival time as the earliest allowed start, but you can no longer rely on sorted starts, so you must first move every room whose end time is <= the arrival into the free heap. Each call stays O(log N).

FOLLOW-UP 2 How do you report the busiest room by total busy time instead of meeting count?

Add the meeting's actual duration to used[room] instead of 1 when you assign it. Rooms are chosen the same way; only the tally changes, and ties still break toward the lowest index.

FOLLOW-UP 3 Rank rooms by total busy time instead of the number of meetings.

Add each meeting's duration (its original length, not the delay) to the room's tally; the choice of room is unchanged.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Simulate room assignments for scheduled
# meetings
#
# This question evaluates scheduling and event-simulation
# skills, including time-ordered processing, resource
# allocation with tie-breaking, and efficient tracking of
# room availability and usage counts using appropriate
# data structures.
#
# FOLLOW-UPS (answered below, in this order):
# 1. What if meetings arrive online and cannot be
# reordered?
# 2. How do you report the busiest room by total busy time
# instead of meeting count?
# 3. Rank rooms by total busy time instead of the number
# of meetings.
#
# INTERVIEW NOTES
# Simulate in start order with two questions per meeting:
# which rooms are free now (lowest index first), and if
# none, which room frees up first (the meeting is delayed
# but keeps its length). Two heaps answer both quickly:
# one of free room ids, one of (end time, room id) for
# busy rooms.
#
# BRUTE FORCE (say it first): Scan all rooms for every
# meeting
# Keep the end time of every room in an array. For each
# meeting in start order, take the lowest-index room that
# has finished by the start; if none, take the room with
# the earliest end (lowest index on ties) and delay the
# meeting. Correct and the oracle; each meeting scans all
# N rooms. Time O(M * N). Space O(N).


import heapq


# BRUTE FORCE (say it first): Scan all rooms for every
# meeting | time O(M * N)
def most_booked_scan(n, meetings):
    end, used = [0] * n, [0] * n
    for start, finish in sorted(meetings):
        free = [r for r in range(n) if end[r] <= start]
        if free:
            r = free[0]
            end[r] = finish
        else:
            r = min(range(n), key=lambda x: (end[x], x))
            end[r] += finish - start
        used[r] += 1
    return used.index(max(used))


# BASE QUESTION (optimal): Two heaps: free rooms and busy
# rooms
# Idea: Process meetings in start order. Move rooms that
# finished by the meeting's start into the free heap
# (lowest index first). If a room is free, use it.
# FOLLOW-UP 3: Rank rooms by total busy time instead of
# the number of meetings.
# Answer: Add each meeting's duration (its original
# length, not the delay) to the room's tally; the choice
# of room is unchanged.
def most_booked_room(n, meetings, by="count"):
    # a meeting takes the lowest-numbered free room; if
    # none is
    # free it waits for the earliest room and keeps its
    # length
    free = list(range(n))
    heapq.heapify(free)
    # busy rooms: (time it becomes free, room)
    busy = []
    used = [0] * n
    for start, end in sorted(meetings):
        # rooms that finished by now are free again
        while busy and busy[0][0] <= start:
            _, room = heapq.heappop(busy)
            heapq.heappush(free, room)
        if free:
            room = heapq.heappop(free)
            heapq.heappush(busy, (end, room))
        else:
            # delayed: starts when the earliest room frees
            # up
            finish, room = heapq.heappop(busy)
            delayed_end = finish + (end - start)
            heapq.heappush(busy, (delayed_end, room))
        # FOLLOW-UP 3: by="time" ranks by total busy time
        # count meetings, or total length (not the delay)
        used[room] += 1 if by == "count" else end - start
    # ties go to the lowest room number
    return used.index(max(used))


if __name__ == "__main__":
    assert most_booked_room(2, [[0, 10], [1, 5], [2, 7],
                                [3, 4]]) == 0
    assert most_booked_room(3, [[1, 20], [2, 10], [3, 5],
                                [4, 9], [6, 8]]) == 1
    # a single room takes everything
    assert most_booked_room(1, [[0, 1], [0, 1]]) == 0
    # by busy time instead of the number of meetings
    meetings = [[0, 100], [1, 2], [2, 3], [3, 4]]
    assert most_booked_room(2, meetings, "count") == 1
    assert most_booked_room(2, meetings, "time") == 0
```

FULL QUESTION TEXT FROM THE SITE

Problem

You are given n meeting rooms labeled 0..n-1 and a list of meetings meetings, where each meeting is [start, end] with start < end. Meetings are processed in increasing order of start time.

Assign each meeting to a room using these rules:

If one or more rooms are free at start , assign the meeting to the free room with the smallest index .
If no rooms are free at start , the meeting must wait until a room becomes free. It is then scheduled in the room that becomes available earliest (break ties by smallest room index). If it waits, its duration stays the same , so if the original duration is end - start and the room becomes free at time t , the meeting runs on [t, t + (end - start)] .

Each time a meeting is assigned to a room (even after waiting), that room’s meeting count increases by 1.

Task

Return the index of the room that hosted the most meetings. If there is a tie, return the smallest index.

FULL QUESTION TEXT FROM THE SITE

Input/Output
Input: n (int), meetings (list of [start,end])
Output: an integer room index
Constraints (reasonable interview assumptions)
1 <= n <= 10^5
1 <= len(meetings) <= 10^5
Times fit in 32-bit signed integer; waiting can increase end times beyond original but still fits in 64-bit.

ALSO REPORTED · 1 more

Assign meetings to rooms with delays | PracHub | Technical Screen

https://prachub.com/interview-questions/assign-meetings-to-rooms-with-delays

STUDY PATH: question 115 of 167 (more practice)

Priority score 2.02 | Intervals and scheduling | asked 287 days ago

HARD | PracHub | Technical Screen | 2 of 4

Minimize travel time with optional meeting point

POSTED 2026-01-06 | 287 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 37.9 | 30 DAYS 33 | PRIORITY 59/228

ABOUT THIS QUESTION

Topic: More practice - Intervals and scheduling

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #743 Network Delay Time, LeetCode #2203 Minimum Weighted Subgraph With the Required Paths

LeetCode match: LC 743 Network Delay Time (multi-source Dijkstra); related LC 2203 (inferred from overview)

Posted on PracHub: 2026-01-06 (updated 2026-05-22)

Original: https://prachub.com/coding-questions/minimize-travel-time-with-optional-meeting-point

QUESTION

Overview: This question evaluates a candidate's understanding of graph shortest-path computation and meeting-point optimization, testing skills in handling weighted edges, aggregating path costs, and reasoning about simultaneous movements on a network.

APPROACH

Run Dijkstra from each traveler's start, then pick the node that minimizes the total (or max, per the statement) travel time; compare against the no-meeting option.

INTERVIEW-DAY PLAN

CLARIFY FIRST

From the scraped solution the setup is two travelers A and B and a destination. Confirm: do they have to travel together after meeting, do they wait for each other (the meeting time is the later arrival), is the graph directed or undirected, and what to return (the minimum time and the meeting node, with ties going to the smallest node, or -1 and -1 if impossible). My copy has only the overview, so verify these.

BRUTE FORCE TO SAY OUT LOUD

Try every node as a meeting point with a fresh shortest path computation each time. O(n * (E log V)).

OPTIMAL IDEA

Run Dijkstra from A, Dijkstra from B, and Dijkstra from the destination on the reversed graph (so you get every node's distance to the destination in one pass). For each node m the time is max(distA[m], distB[m]) + distToDest[m]. Take the minimum, with ties to the smallest node. Compare with the no-meeting option if each can also go directly. O((V + E) log V).

INTERVIEW-DAY PLAN

DRY-RUN THESE CASES

A and B start at the same node, a node that only one of them can reach, a directed graph where the way back is longer, two nodes with equal time.

LIKELY FOLLOW-UPS

More travelers (Dijkstra from each, take the max), minimize total distance instead of arrival, or an optional meeting point.

PITFALLS

Forgetting to reverse the graph for the destination distances in a directed graph. Using sum instead of max for waiting.

HOW TO THINK ABOUT IT

Break it into shortest paths from three places: from A, from B, and into the destination (a Dijkstra on the reversed graph if directed). For each node m the cost is max(distA[m], distB[m]) + dist(m to destination); take the minimum. Three single-source searches replace trying every meeting node with its own search.

Basis: Written from the scraped solution (confirm it): two travellers A and B, a destination; they may meet at a node (the meeting happens when the later one arrives) and then travel together to the destination; return [best time, best node] with ties to the smallest node, or [-1, -1].

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(n, edges, A, B, dest):
    d = floyd_warshall(n, edges)  # all pairs
    best, best_m = INF, None
    for m in range(n):
        cost = max(d[A][m], d[B][m]) + d[m][dest]
        if cost < best:
            best, best_m = cost, m
    return best
```

TIME	SPACE
O(V^3)	O(V^2)

WHY IT IS SLOW

All-pairs distances cost O(V^3) when only three single-source distances are needed. Dijkstra from A, from B and into the destination is O((V + E) log V).

OPTIMAL SOLUTION · Three Dijkstra runs (from A, from B, and into the destination)

TIME	SPACE
O((V + E) log V)	O(V + E)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 More than two travellers.

Run one Dijkstra per traveller and one from the destination, then for each node take the maximum arrival time (if they wait for each other) or the sum of distances (if the cost is total distance) plus the shared leg. K + 1 searches.

FOLLOW-UP 2 Meeting is optional.

Also compute the cost with no meeting (each goes straight to the destination: max(distA[dest], distB[dest])) and take the smaller. Meeting at the destination itself already covers it when the cost is the later arrival.

FOLLOW-UP 3 Edge weights change over time (traffic).

Dijkstra still works if the graph is FIFO (leaving later never arrives earlier); each edge's cost becomes a function of the departure time, evaluated when you relax it.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Minimize travel time with optional
# meeting point
#
# This question evaluates a candidate's understanding of
# graph shortest-path computation and meeting-point
# optimization, testing skills in handling weighted edges,
# aggregating path costs, and reasoning about simultaneous
# movements on a network.
#
# FOLLOW-UPS (answered below, in this order):
# 1. More than two travellers.
# 2. Meeting is optional.
# 3. Edge weights change over time (traffic).
#
# INTERVIEW NOTES
# Break it into shortest paths from three places: from A,
# from B, and into the destination (a Dijkstra on the
# reversed graph if directed). For each node m the cost is
# max(distA[m], distB[m]) + dist(m to destination); take
# the minimum. Three single-source searches replace trying
# every meeting node with its own search.
#
# BRUTE FORCE (say it first): Floyd-Warshall all-pairs,
# then try every meeting node
# Compute the shortest distance between every pair of
# nodes, then for each node m take max(d[A][m], d[B][m]) +
# d[m][dest]. Simple and the oracle; cubic in the number
# of nodes. Time O(V^3). Space O(V^2).


import heapq


# HELPERS for the brute force
INF = float("inf")


# BRUTE FORCE (say it first): Floyd-Warshall all-pairs,
# then try every meeting node | time O(V^3)
def meeting_point_floyd(n, edges, a, b, dest, directed):
    INF = float("inf")
    d = [
        [0 if i == j else INF for j in range(n)]
        for i in range(n)
    ]
    for u, v, w in edges:
        d[u][v] = min(d[u][v], w)
        if not directed:
            d[v][u] = min(d[v][u], w)
    for k in range(n):
        for i in range(n):
            for j in range(n):
                if d[i][k] + d[k][j] < d[i][j]:
                    d[i][j] = d[i][k] + d[k][j]
    best = (INF, -1)
    for m in range(n):
        t = max(d[a][m], d[b][m]) + d[m][dest]
        if t < best[0]:
            best = (t, m)
    return (
        [-1, -1]
        if best[0] == INF
        else [best[0], best[1]]
    )


# HELPERS for the base question
def dijkstra(graph, source):
    dist = [INF] * len(graph)
    dist[source] = 0
    heap = [(0, source)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        for v, w in graph[u]:
            if d + w < dist[v]:
                dist[v] = d + w
                heapq.heappush(heap, (d + w, v))
    return dist


def build(n, edges, directed):
    # forward graph, and the reversed graph for "to dest"
    fwd = [[] for _ in range(n)]
    rev = [[] for _ in range(n)]
    for u, v, w in edges:
        fwd[u].append((v, w))
        rev[v].append((u, w))
        if not directed:
            fwd[v].append((u, w))
            rev[u].append((v, w))
    return fwd, rev


# BASE QUESTION (optimal): Three Dijkstra runs (from A,
# from B, and into the destination)
# Idea: distA and distB come from forward searches. The
# distance from every node to the destination is a search
# from the destination on the reversed graph (the same
# graph if undirected).
def meeting_point(n, edges, a, b, dest, directed=False):
    # both start at the same time, meet at node m, wait
    # for
    # each other, then travel together to dest. Meeting at
    # dest itself covers the "no meeting" plan.
    fwd, rev = build(n, edges, directed)
    da, db = dijkstra(fwd, a), dijkstra(fwd, b)
    # distance from every node TO dest = search the
    # reverse graph
    to_dest = dijkstra(rev, dest)
```

```
    best = (INF, -1)
    for m in range(n):
        total = max(da[m], db[m]) + to_dest[m]
        if total < best[0]:
            best = (total, m)
    return [-1, -1] if best[0] == INF else list(best)


# FOLLOW-UP 1: More than two travellers.
# Answer: Run one Dijkstra per traveller and one from the
# destination, then for each node take the maximum arrival
# time (if they wait for each other) or the sum of
# distances (if the cost is total distance) plus the
# shared leg.
def meeting_point_many(n, edges, starts, dest,
                       directed=False):
    # k travellers: one search each, plus one from dest
    fwd, rev = build(n, edges, directed)
    dists = [dijkstra(fwd, s) for s in starts]
    to_dest = dijkstra(rev, dest)
    best = (INF, -1)
    for m in range(n):
        wait = max(d[m] for d in dists)
        if wait + to_dest[m] < best[0]:
```

```
            best = (wait + to_dest[m], m)
    return [-1, -1] if best[0] == INF else list(best)


if __name__ == "__main__":
    # a star: both travellers reach the hub 2, then go to
    # 3
    edges = [(0, 2, 1), (1, 2, 2), (2, 3, 4), (0, 3, 9),
             (1, 3, 9)]
    # meet at 2: max(1, 2) + 4 = 6; going alone costs 9
    assert meeting_point(4, edges, 0, 1, 3) == [6, 2]
    # nobody can reach the destination
    short = [(0, 1, 1)]
    assert meeting_point(3, short, 0, 1, 2) == [-1, -1]
    # same start: they simply walk together
    assert meeting_point(4, edges, 0, 0, 3)[0] == 5
    # three travellers
    got = meeting_point_many(4, edges, [0, 1, 1], 3)
    assert got == [6, 2]
    # directed roads: reverse edges are not usable
    one_way = [(0, 1, 1), (1, 2, 1)]
    walk = meeting_point(3, one_way, 0, 1, 2, True)
    assert walk == [2, 1]
```

FUNCTION AND TEST CASES

Signature

```
def solution(n: int, edges: list[list[int]], startA: int,
startB: int, dest: int, directed: bool = False) -> list[in
t]:
```

Sample call

```
solution(4, [(0, 1, 2), (1, 2, 1), (1, 3, 2), (2, 3, 2)],
0, 2, 3, False)
```

Test cases

Input	Expected
(4, [(0, 1, 2), (1, 2, 1), (1, 3, 2), (2, 3, 2)], 0, 2, 3, False)	[4, 1]
(4, [(0, 1, 2), (2, 1, 3), (1, 3, 2), (0, 3, 10), (2, 3, 5)], 0, 2, 3, True)	[5, 1]

STUDY PATH: question 116 of 167 (more practice)

Priority score 1.69 | Intervals and scheduling | asked 316 days ago

HARD | IN APP LC 218 | PracHub | Technical Screen | 3 of 4

Compute city skyline outline

POSTED 2025-12-08 | 316 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 29.8 | 30 DAYS 0 | PRIORITY 125/228

ABOUT THIS QUESTION

Topic: More practice - Intervals and scheduling

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #218 The Skyline Problem

LeetCode match: LC 218 The Skyline Problem

Posted on PracHub: 2025-12-08 (updated 2026-06-26)

Original: https://prachub.com/coding-questions/compute-city-skyline-outline

QUESTION

You are given a list of rectangular buildings in a 2D city skyline. Each building is represented by three integers `[L, R, H]`:

- `L`: the x-coordinate of the left edge of the building,
- `R`: the x-coordinate of the right edge of the building (`L < R`),
- `H`: the height of the building. All buildings stand on a flat horizontal line (y = 0) and extend vertically up to height `H`. Buildings may overlap in the x-dimension. Your task is to compute the *skyline* formed by these buildings, as seen from a distance. The skyline should be represented as a list of **key points** `[x, height]` that describe the outer contour of the union of all buildings. A **key point** is a point where the height of the skyline changes. The output list must satisfy all of the following:

1. The list is sorted by increasing **x**.
2. No two consecutive points have the same height (i.e., the height must change at each key point).
3. The first point has a height greater than 0.
4. The last point has height 0, marking the end of the skyline.

Input

- An integer **n** — the number of buildings.
- A list of **n** buildings, where each building is given as a triple `[L, R, H]` of integers. You may assume:
- `1 <= n <= 10^4`
- `0 <= L < R <= 10^9`
- `1 <= H <= 10^9`

Output

Return a list of key points representing the skyline. Each key point is a pair `[x, height]` (or a 2-element array/tuple) of integers.

Example

Input:

- `n = 3` - buildings = `[[2, 9, 10], [3, 7, 15], [5, 12, 12]]` One valid output:

QUESTION

- `[[2, 10], [3, 15], [7, 12], [12, 0]]` Explanation (informal):
- From x = 2 to 3, the highest building has height 10.
- From x = 3 to 7, the highest building has height 15.
- From x = 7 to 12, the remaining highest building has height 12.
- After x = 12, there are no buildings, so the height drops to 0. Design an algorithm to compute the skyline key points for the given buildings.

APPROACH

Sweep line over building edges with a max-heap of active heights (lazy deletion); emit a point whenever the max height changes. O(n log n). Divide and conquer also works.

HOW TO THINK ABOUT IT

The skyline height at x is the tallest building covering x, and it can only change at a building's left or right edge. So sweep over those x positions in order, keep the active buildings in a max-heap by height, and report a key point whenever the maximum changes. The details that matter are the event order at the same x and removing finished buildings lazily.

Basis: LeetCode 218, The Skyline Problem.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(buildings):
    xs = set()
    for L, R, H in buildings:
        xs |= {L, R}
    out, prev = [], 0
    for x in sorted(xs):
        h = 0
        for L, R, H in buildings:
            if L <= x < R:
                h = max(h, H)
        if h != prev:
            out.append([x, h])
            prev = h
    return out
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Each edge scans all buildings, O(n^2). A sweep with a max-heap of active buildings and lazy deletion is O(n log n).

OPTIMAL SOLUTION · Sweep line with a max-heap and lazy deletion

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Divide and conquer instead of a heap.

Build the skyline of each half and merge the two by sweeping both lists with the current height of each; the merged height is the max of the two. T(n) = 2T(n/2) + O(n) = O(n log n), same as the heap.

FOLLOW-UP 2 Coordinates up to 10^9 or buildings of a very large count.

Only the edge coordinates matter, so nothing depends on the coordinate range. The sweep is O(n log n) in the number of buildings; use a balanced tree or a heap with lazy deletion for the active set.

FOLLOW-UP 3 Output the area under the skyline.

Sum height * (next x - x) over consecutive key points: one extra pass over the output.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Compute city skyline outline
#
# You are given a list of rectangular buildings in a 2D
# city skyline. Each building is represented by three
# integers [L, R, H]: - L: the x-coordinate of the left
# edge of the building, - R: the x-coordinate of the right
# edge of the building (L < R), - H: the height of the
# building. All buildings stand on a flat horizontal line
# (y = 0) and extend vertically up to height H. Buildings
# may overlap in the x-dimension. Your task is to compute
# the *skyline* formed by these buildings, as seen from a
# distance. The skyline should be represented as a list of
# **key points** [x, height] that describe the outer
# contour of the union of all buildings. A **key point**
# is a point where the height of the skyline changes. The
# output list must satisfy all of the following: 1. The
# list is sorted by increasing x. 2. No two consecutive
# points have the same height (i.e., the height must
# change at each key point). 3.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Divide and conquer instead of a heap.
# 2. Coordinates up to 10^9 or buildings of a very large
# count.
# 3. Output the area under the skyline.
#
# INTERVIEW NOTES
# The skyline height at x is the tallest building covering
# x, and it can only change at a building's left or right
# edge. So sweep over those x positions in order, keep the
# active buildings in a max-heap by height, and report a
# key point whenever the maximum changes. The details that
# matter are the event order at the same x and removing
# finished buildings lazily.
#
# BRUTE FORCE (say it first): Compute the height at every
# edge coordinate
# Collect all edge x-coordinates in order. At each one the
# height is the maximum height among buildings with L <= x
# < R. Output a point where the height differs from the
# previous one. Correct and the oracle; each coordinate
# scans all buildings. Time O(n^2). Space O(n).


import heapq


# BRUTE FORCE (say it first): Compute the height at every
# edge coordinate | time O(n^2)
def skyline_brute(buildings):
    xs = sorted(
        {x for l, r, h in buildings for x in (l, r)}
    )
    out, prev = [], 0
    for x in xs:
        h = max(
            (
                hh
                for l, r, hh in buildings
                if l <= x < r
            ),
            default=0,
        )
        if h != prev:
            out.append([x, h])
            prev = h
    return out


# HELPERS for follow-up 1
def merge(a, b):
    # divide and conquer: merge two skylines with the
    # current
    # height of each; the result height is their maximum
    out, i, j, ha, hb = [], 0, 0, 0, 0
    while i < len(a) or j < len(b):
        take_a = i < len(a) and (j == len(b)
                                 or a[i][0] <= b[j][0])
        if take_a:
            x, ha = a[i]
            i += 1
        else:
            x, hb = b[j]
            j += 1
        # when both change at the same x, wait for the
        # second
        if i < len(a) and a[i][0] == x:
            continue
        if j < len(b) and b[j][0] == x:
            continue
        top = max(ha, hb)
        if not out or out[-1][1] != top:
            out.append([x, top])
    return out


# BASE QUESTION (optimal): Sweep line with a max-heap and
# lazy deletion
# Idea: Make events (x, -height) for left edges and (x, 0,
# right) for right edges, sorted so that at the same x
```

```
# taller starts come first and ends come after starts.
# Keep a heap of (-height, right edge).
def skyline(buildings):
    # buildings are [left, right, height]
    # start events carry -height so taller starts sort
    # first;
    # end events carry 0
    events = sorted([(l, -h, r) for l, r, h in buildings]
                    + [(b[1], 0, 0) for b in buildings])
    out = []
    # the active buildings: (-height, right edge), ground
    # first
    heap = [(0, float("inf"))]
    for x, neg_h, right in events:
        # buildings that ended at or before x are gone
        while heap[0][1] <= x:
            heapq.heappop(heap)
        if neg_h:
            heapq.heappush(heap, (neg_h, right))
        top = -heap[0][0]
        # a key point only where the height changes
        if not out or out[-1][1] != top:
            out.append([x, top])
    return out


# FOLLOW-UP 1: Divide and conquer instead of a heap.
# Answer: Build the skyline of each half and merge the two
# by sweeping both lists with the current height of each;
# the merged height is the max of the two. T(n) = 2T(n/2)
# + O(n) = O(n log n), same as the heap.
def skyline_dc(buildings):
    if len(buildings) == 1:
        l, r, h = buildings[0]
        return [[l, h], [r, 0]]
    if not buildings:
        return []
    mid = len(buildings) // 2
    return merge(skyline_dc(buildings[:mid]),
                 skyline_dc(buildings[mid:]))


# FOLLOW-UP 3: Output the area under the skyline.
# Answer: Sum height * (next x - x) over consecutive key
# points: one extra pass over the output.
def area_under(points):
    # sum of height * width between consecutive key points
    return sum(h * (points[i + 1][0] - x)
               for i, (x, h) in enumerate(points[:-1]))


if __name__ == "__main__":
    b = [[2, 9, 10], [3, 7, 15], [5, 12, 12],
         [15, 20, 10], [19, 24, 8]]
    want = [[2, 10], [3, 15], [7, 12], [12, 0], [15, 10],
            [20, 8], [24, 0]]
    assert skyline(b) == want
    assert skyline_dc(b) == want
    flat = skyline([[0, 2, 3], [2, 5, 3]])
    assert flat == [[0, 3], [5, 0]]
    assert skyline([]) == []
    # 10*1 + 15*4 + 12*5 + 10*5 + 8*4 = 212
    assert area_under(want) == 212
    assert skyline([[1, 2, 1], [1, 2, 2], [1, 2, 3]]) == [
        [1, 3], [2, 0]]
```

FUNCTION AND TEST CASES

Sample call

```
getSkyline(3, [[2, 9, 10], [3, 7, 15], [5, 12, 12]])
```

Test cases

Input	Expected
(3, [[2, 9, 10], [3, 7, 15], [5, 12, 12]])	[[2, 10], [3, 15], [7, 12], [12, 0]]
(1, [[0, 2, 3]])	[[0, 3], [2, 0]]

STUDY PATH: question 117 of 167 (more practice)

Priority score 1.5 | Intervals and scheduling | asked 169 days ago

HARD | PracHub | Technical Screen | 4 of 4

Compute Turnstile Crossing Times

POSTED 2026-05-04 | 169 DAYS AGO | LAST 6 MONTHS

STUDY: TALK **NO LEETCODE FREQUENCY** **PRIORITY 176/228**

ABOUT THIS QUESTION

Topic: More practice - Intervals and scheduling

Optimal answer from: the interview site's own solution

LeetCode match: LeetCode 'Time Taken to Cross the Door' (exact number unverified) (inferred from overview)

Posted on PracHub: 2026-05-04 (updated 2026-05-10)

Original: https://prachub.com/coding-questions/compute-turnstile-crossing-times

QUESTION

Overview: This question evaluates proficiency in queue simulation, event-driven scheduling, and handling priority and tie-breaking rules while achieving linear time complexity.

APPROACH

Two queues (entering, exiting) ordered by arrival; simulate time with the direction-priority and tie-break rules, jumping the clock when idle. O(n).

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask: how many people can pass per second, which direction goes first when both are waiting, whether the rule depends on what happened in the previous second, and how ties on arrival time break (by id). Clarify what to return (time per person).

BRUTE FORCE TO SAY OUT LOUD

Simulate every second from the earliest to the latest time, checking both queues. Correct but its cost grows with the size of the timestamps, not the number of people.

OPTIMAL IDEA

Keep two FIFO queues, enter and exit, and a variable for what the turnstile did in the previous second. Each step apply the priority rule to pick a queue. When both queues are empty, jump the clock to the next arrival and reset the previous-state to idle if at least one second was skipped. That makes the simulation O(n) after sorting.

DRY-RUN THESE CASES

Two people arriving at the same second with opposite directions. A long idle gap followed by a simultaneous pair. All enters, then all exits. A person who arrives while the turnstile was just used by the other direction.

LIKELY FOLLOW-UPS

INTERVIEW-DAY PLAN

Ask what happens when the idle jump leaves exactly one empty second versus more. Count that the 'previous second' state can be none, not only enter or exit.

PITFALLS

Resetting state only when the gap is two or more seconds. Forgetting to push arrivals that happened during the jump. Comparing queues by id instead of arrival order.

HOW TO THINK ABOUT IT

A scheduling rule that looks at 'what happened last second' is a state machine: write the state down (previous direction: enter, exit, none) before coding. If time values can be huge, simulate events, not seconds: jump the clock to the next arrival when both queues are empty, and remember an idle gap resets the state.

Basis: LeetCode 2534, Time Taken to Cross the Door. Assumed rules (confirm them): one person per second; if an enter and an exit compete, the previous second decides (enter if it was an enter, else exit, and exit if the door was idle).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(arrivals):
    t, prev = 0, None
    enter_q, exit_q = deque(), deque()
    while pending(arrivals, enter_q, exit_q):
        add_arrivals(t, arrivals, enter_q,
                     exit_q)
        who = pick_by_priority(prev, enter_q,
                               exit_q)
        if who is not None:
            passed[who] = t
            prev = direction_of(who)
        t += 1
    return passed
```

TIME	SPACE
O(T + n) where T is the largest timestamp	**O(n)**

WHY IT IS SLOW

A timestamp of 10^9 means a billion iterations even if nothing happens. Jumping to the next arrival when both queues are empty is O(n log n).

OPTIMAL SOLUTION · Event-driven simulation with two queues

TIME	SPACE
O(n log n)	**O(n)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The priority when the turnstile was idle is different in this version of the question.

Keep the same loop and make the idle-state winner a parameter. Only the line that picks the queue changes, which is why the state machine should be written first.

FOLLOW-UP 2 Timestamps are up to 10^9.

That is exactly why the simulation jumps between events: the loop runs once per person, so the answer is O(n log n) no matter how large the times are.

FOLLOW-UP 3 Two turnstiles or a capacity of two people per second.

Run the same pick-a-queue step twice per tick (or per turnstile) and carry a previous-state variable per turnstile. Decide whether the two turnstiles share the queues.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Compute Turnstile Crossing Times
#
# This question evaluates proficiency in queue simulation,
# event-driven scheduling, and handling priority and tie-
# breaking rules while achieving linear time complexity.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The priority when the turnstile was idle is different
# in this version of the question.
# 2. Timestamps are up to 10^9.
# 3. Two turnstiles or a capacity of two people per
# second.
#
# INTERVIEW NOTES
# A scheduling rule that looks at 'what happened last
# second' is a state machine: write the state down
# (previous direction: enter, exit, none) before coding.
# If time values can be huge, simulate events, not
# seconds: jump the clock to the next arrival when both
# queues are empty, and remember an idle gap resets the
# state.
#
# BRUTE FORCE (say it first): Simulate every second
# Walk t = 0, 1, 2, ... adding arrivals to two FIFO queues
# and letting one person through per second by the rule.
# It is easy to verify, but it spends time on every empty
# second, so a timestamp of 10^9 is hopeless. Time O(T +
# n) where T is the largest timestamp. Space O(n).


from collections import deque


# BRUTE FORCE (say it first): Simulate every second | time
# O(T + n) where T is the largest timestamp
def turnstile_brute(events):
    n = len(events)
    order = sorted(
        range(n), key=lambda i: (events[i][0], i)
    )
    res, ent, ext, prev, t, k = (
        [None] * n,
        deque(),
        deque(),
        None,
        0,
        0,
    )
    while any(r is None for r in res):
        while k < n and events[order[k]][0] <= t:
            (
                ent
                if events[order[k]][1] == "enter"
                else ext
            ).append(order[k])
            k += 1
        if ent and ext:
            q = ent if prev == "enter" else ext
        else:
            q = ent or ext or None
        if q is None:
            prev = None
        else:
            p = q.popleft()
            res[p], prev = t, events[p][1]
        t += 1
    return res


# BASE QUESTION (optimal): Event-driven simulation with
# two queues
# Idea: Sort arrivals once. When both queues are empty,
# jump straight to the next arrival time; if the jump
# leaves at least one idle second, reset the previous
# state to 'none'.
# FOLLOW-UP 1: The priority when the turnstile was idle is
# different in this version of the question.
# Answer: Keep the same loop and make the idle-state
# winner a parameter. Only the line that picks the queue
# changes, which is why the state machine should be
# written first.
def turnstile_times(events, idle_first="exit"):
    # events[i] = (arrival time, "enter" or "exit");
    # the result is the second each person goes through
    n = len(events)
    order = sorted(range(n),
                   key=lambda i: (events[i][0], i))
    lines = {"enter": deque(), "exit": deque()}
    result = [0] * n
    t, i, prev, last_used = 0, 0, None, -10
    while i < n or lines["enter"] or lines["exit"]:
        if not lines["enter"] and not lines["exit"]:
            # nobody is waiting: jump to the next arrival
            t = max(t, events[order[i]][0])
            # an idle second forgets the last direction
            if t > last_used + 1:
                prev = None
        # everyone who has arrived by now joins a line
        while i < n and events[order[i]][0] <= t:
            person = order[i]
            lines[events[person][1]].append(person)
            i += 1
        if lines["enter"] and lines["exit"]:
            # FOLLOW-UP 1: idle tie-break set by
            # idle_first
            # both lines: keep the direction used last
            # second,
            # and use idle_first after an idle gap
            want = prev or idle_first
        else:
            want = "enter" if lines["enter"] else "exit"
```

```
        person = lines[want].popleft()
        result[person] = t
        prev, last_used = want, t
        t += 1
    return result


if __name__ == "__main__":
    both = [(0, "enter"), (0, "exit")]
    # idle tie: the exit goes first
    assert turnstile_times(both) == [1, 0]
    # a version where enter wins the idle tie
    assert turnstile_times(both, "enter") == [0, 1]
    # the direction in use keeps going
    keep = [(0, "enter"), (0, "enter"), (1, "exit")]
    assert turnstile_times(keep) == [0, 1, 2]
    # an idle second resets the state
    gap = [(0, "enter"), (10, "enter"), (10, "exit")]
    assert turnstile_times(gap) == [0, 11, 10]
    # huge timestamps are no problem: one loop per person
    far = [(10 ** 9, "enter"), (10 ** 9, "exit")]
    assert turnstile_times(far) == [10 ** 9 + 1, 10 ** 9]
    assert turnstile_times([]) == []
```

FUNCTION AND TEST CASES

Signature

```
def solution(time: list[int], direction: list[int]) -> lis
t[int]:
SAMPLE INPUT
[[0,0,1,5],[0,1,1,0]]
```

Test cases

Input	Expected
[[0, 0, 1, 5], [0, 1, 1, 0]]	[2, 0, 1, 5]
[[0, 0, 0], [0, 1, 0]]	[1, 0, 2]